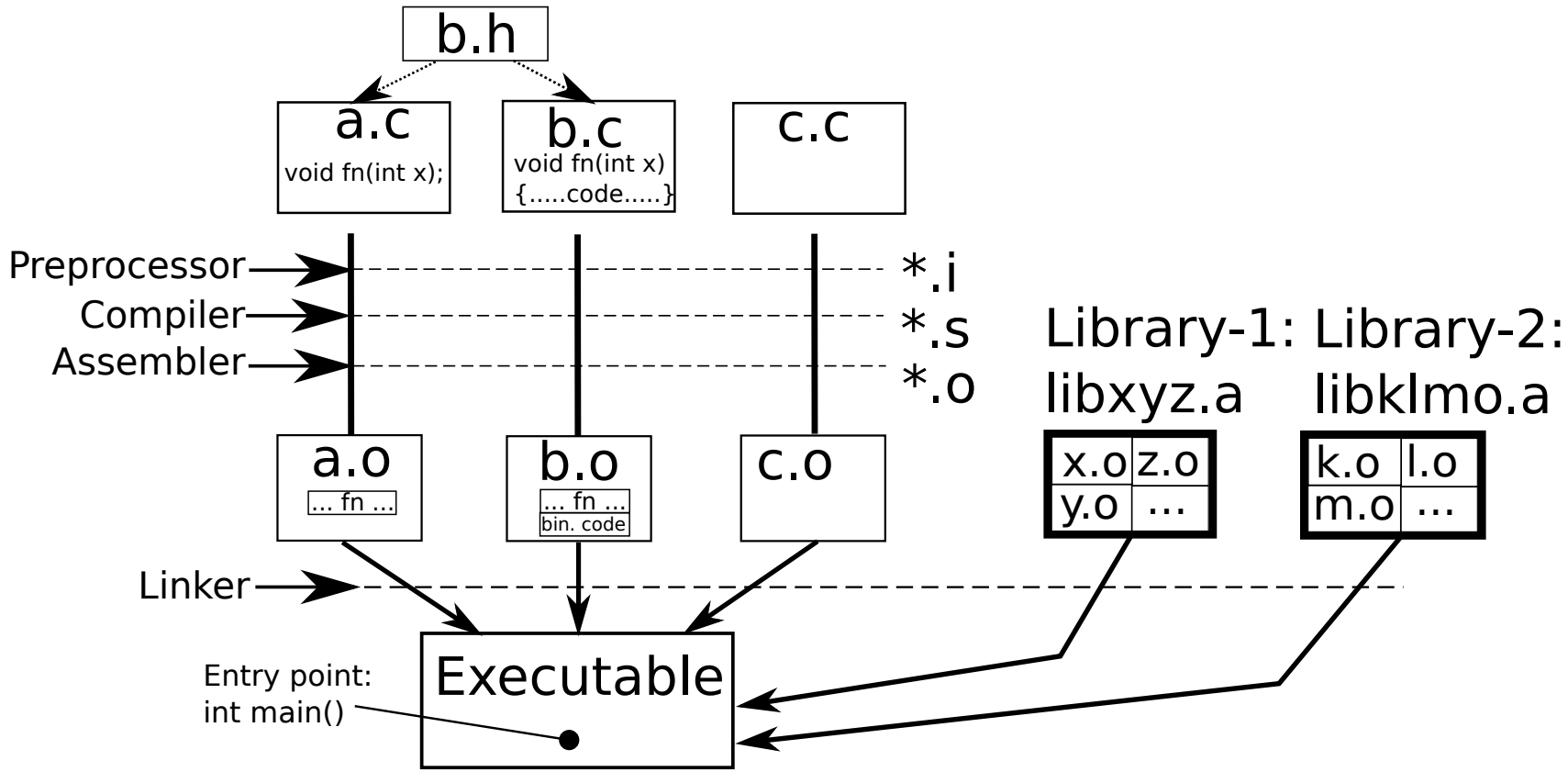
C++ — Додаток. Make і Makefile





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)
https://www.facebook.com/oleg.farenyuk
indrekis@gmail.com

Роздільна компіляція



Програма з декількох файлів — 1

main_file.cpp

```
#include <iostream>

#include "file1.h"
#include "file2.h"

int main()
{
    std::cout << "Hello world from main." << std::endl;

    function_from_file1();

    function_from_file2();

    return 0;
}
```

Програма з декількох файлів — 2

file1.h

```
#ifndef FILE1_HEADER
#define FILE1_HEADER

void function_from_file1();

#endif // FILE1_HEADER
```

file1.cpp

```
#include <iostream>

#include "file1.h"

void function_from_file1()
{
    std::cout << "Hello world from __file__ 1." << std::endl;
}
```

Програма з декількох файлів — 3

file2.h

```
#ifndef FILE2_HEADER
#define FILE2_HEADER

void function_from_file2();

#endif // FILE2_HEADER
```

file2.cpp

```
#include <iostream>

#include "file2.h"

void function_from_file2()
{
        std::cout << "Hello world from ~~~file~~~ 2." << std::endl
}
```

Компілюємо — прямолінійний підхід

- ```
  g++ -c main_file.cpp -o  main_file.o
  g++ -c file1.cpp -o  file1.o
  g++ -c file2.cpp -o  file2.o
  g++ file1.o file2.o main_file.o -o program
  ```
- **Зауваження:** У прикладі вище, для виклику компонувальника, використано той же компілятор, g++ — C++ компілятор із GCC. Це доволі поширена практика. Сам лінкер (ld) вимагає складної, часто неочевидної, комбінації опцій. Щоб полегшити життя розробникам, компілятор вміє розпізнати, коли його викликають для компонування, і викликає лінкер із правильним їх набором.
  - ◇ Як би там не було, не дуже зручно.
- Конкретно тут можна схитрувати:

  ```
  g++ main_file.cpp file1.cpp file2.cpp -o program
  ```
  - ◇ Однак цей спосіб далеко не універсальний і має ряд недоліків. Наприклад, всі файли компілюються із тим самим набором опцій. Крім того, навіть якщо змінюється один файл, перекомпільовуються всі.
  ```

Відступ: безпосередній виклик лінкера

- Мінімалістичний приклад для компіляції коду на C[1], MinGW:

```
ld /mingw64/x86_64-w64-mingw32/lib/crt2.o \
   /mingw64/lib/gcc/x86_64-w64-mingw32/10.1.0/crtbegin.o \
  -L/mingw64/lib/gcc/x86_64-w64-mingw32/10.1.0 \
  -L/mingw64/x86_64-w64-mingw32/lib critical_function.o \
   file1.o file2.o main_file.o -o program \
  -lmingw32 -lgcc -lmingwex -lmsvcrt -lkernel32 -lmsvcrt \
   /mingw64/x86_64-w64-mingw32/lib/default-manifest.o \
   /mingw64/lib/gcc/x86_64-w64-mingw32/10.1.0/crtend.o
```

- Для Linux:

```
ld -dynamic-linker \
  /lib64/ld-linux-x86-64.so.2
  /usr/lib/x86_64-linux-gnu/Scrt1.o
  /usr/lib/x86_64-linux-gnu/crti.o \
  /usr/lib/gcc/x86_64-linux-gnu/9/crtbeginS.o \
   file1.o file2.o main_file.o -o program     -lc \
  /usr/lib/gcc/x86_64-linux-gnu/9/crtendS.o \
  /usr/lib/x86_64-linux-gnu/crtn.o
```

- На практиці gcc використовуватиме помітно складніший командний рядок для лінкера, побачити який можна так:

```
gcc --verbose <.......>
```

[1] Для C++ – ще більший, тут не влазиться

make

make — приклад системи автоматичної побудови програм.

- Системи автоматичної побудови програм — програми, призначені керувати побудовою інших програм.
- Структура цільової програми описується тим чи іншим способом, після чого, компіляція може здійснюватися викликом цієї системи.
- Make — напевне, найбільш поширена така система.
- Опис програми для неї здійснюється написанням спеціальних файлів, Makefile.

Виклик make

- Найпростіший спосіб викликати make — просто набрати "make" в командному рядку, знаходячись в директорії проекту. Якщо там знаходиться файл з іменем `Makefile.` або[2] `makefile.`, без розширення, то вона вважатиме, що в ньому є опис правил побудови проекту[3].
- Альтернативні, більш універсальні, форми виклику наступні:

```
make -f my_makefile_file.ext
make --file=my_makefile_file.ext
make --makefile=my_makefile_file.ext
```

- Один Makefile може містити опис декількох задач — цілей компіляції. Вказують їх так:

```
make targename
make -f my_makefile_file.ext targename
```

[2] Пам'ятаємо, що для багатьох файлових систем, великі та малі літери вважаються різними.

[3] Часто припустимі й інші імена по замовчуванню. Слід звірятися з документацією конкретної версії make.

Інші опції make

Декілька важливих опцій[4]:

- `-j N` — компілювати, використовуючи N потоків. Вимагає дотримання ряду вимог, щоб працювати коректно.
- `-n`, `--dry-run` — тестовий запуск, не виконувати жодної команди, лише друкувати ті, які виконалися б без цієї опції.
- `-k`, `--keep-going` — продовжувати виконання, не зважаючи на помилки. Нормальною поведінкою make є зупинити компіляцію після першої помилки.

За рештою — див. документацію конкретної версії make.

[4] Лише частина із можливих форм, детальніше див. документацію, або, хоча б "make --help".

Найпростіший Makefile

```
hellomake: main_file.cpp file1.cpp file2.cpp
    g++ main_file.cpp file1.cpp file2.cpp -o hellomake
```

УВАГА! Перед командою, в даному випадку, назвою компілятора — g++, має стояти **СИМВОЛ ТАБУЛЯЦІЇ**. Інакше make буде казати щось типу:

```
Makefile:2: *** missing separator.  Stop.
```

hellomake — **ціль компіляції**. Те, що йде **після двокрапки — її залежності**, які мають бути актуальними, або які слід побудувати, щоб можна було зайнятися нею. В даному прикладі це просто файли з текстом програми, написані людиною, тому make перейде зразу до побудови hellomake.

Загальний вигляд цілі для make такий:

```
target: dependencies
[tab] command options
```

Компіляція кожного файла окремо

Попередній makefile має помітну частину тих же недоліків, що й безпосередня компіляція. Зокрема, кожного разу всі файли перекомпільовуватимуться. Виправимо це:

```
all: hellomake

hellomake: main_file.o file1.o file2.o
    g++ main_file.o file1.o file2.o -o hellomake

main_file.o: main_file.cpp
    g++ -c main_file.cpp

file1.o: file1.cpp
    g++ -c file1.cpp

file2.o: file2.cpp
    g++ -c file2.cpp
```

Нагадую про те, що перед g++ всюди вище **СИМВОЛ ТАБУЛЯЦІЇ**!

- Якщо make не передано явних цілей, виконується та, яка трапилася першою.
- Ціль може не містити команд. Вона вважається "задоволеною" (fulfilled), якщо задоволені всі цілі, від яких вона залежить.
- Тобто, у прикладі вище, all буде задоволеним, якщо успішно скомпілюється ціль hellomake. Вона, у свою чергу, залежить від main_file.o, file1.o, file2.o. Кожному об'єктному файлу призначена своя ціль.
- Компіляція "з нуля" при цьому виглядатиме так:

```
[...]$ make -f makefile.2
g++ -c main_file.cpp
g++ -c file1.cpp
g++ -c file2.cpp
g++ main_file.o file1.o file2.o -o hellomake
```

- Якщо, після цього, змінити лише file2.c, і перебудувати, отримаємо:

```
[...]$ make -f makefile.2
g++ -c file2.cpp
g++ main_file.o file1.o file2.o -o hellomake
```

Файли заголовків

- Однак, якщо змінити файли заголовків, перекомпіляції не відбудеться.
- Як з цим боротися систематично, на прикладі GNU make і gcc/g++, розглянемо трішки пізніше.
- Найпростіше ж рішення — вказати їх безпосередньо:

```
all: hellomake

hellomake: main_file.o file1.o file2.o
    g++ main_file.o file1.o file2.o -o hellomake

main_file.o: main_file.cpp file1.h file2.h
    g++ -c main_file.cpp

file1.o: file1.cpp  file1.h
    g++ -c file1.cpp

file2.o: file2.cpp  file2.h
    g++ -c file2.cpp
```

Ціль clean

Зручною є також ціль для повної очистки слідів компіляції. Її, традиційно, називають clean:

```
all: hellomake

hellomake: main_file.o file1.o file2.o
    g++ main_file.o file1.o file2.o -o hellomake

main_file.o: main_file.cpp file1.h file2.h
    g++ -c main_file.cpp

file1.o: file1.cpp  file1.h
    g++ -c file1.cpp

file2.o: file2.cpp  file2.h
    g++ -c file2.cpp

.PHONY : clean
clean:
    rm -rf *.o hellomake
```

"`.PHONY : clean`" служить, щоб повідомити make — в результаті обробки цієї цілі не з'явиться файл з іменем clean.

Коментарі

Як і більшість інших мов програмування (а засоби make, в певному сенсі, можна вважати такою мовою), синтаксис make дозволяє створювати коментарі.

```
# This is makefile
all: hellomake # default target
```

Змінні

Описаний вище makefile цілком справляється із своєю задачею, однак є занадто конкретним. Спробуємо спростити його подальшу модифікацію.

```
CXX = g++

all: hellomake

hellomake: main_file.o file1.o file2.o
    $(CXX) main_file.o file1.o file2.o -o hellomake

main_file.o: main_file.cpp file1.h file2.h
    $(CXX) -c main_file.cpp

file1.o: file1.cpp  file1.h
    $(CXX) -c file1.cpp

file2.o: file2.cpp  file2.h
    $(CXX) -c file2.cpp

.PHONY : clean
clean:
    rm -rf *.o hellomake
```

Тему із змінними можна розвивати далі. Популярними є змінні із опціями компілятора, списком об'єктних файлів, тощо:

```
CXX = g++
CXXFLAGS = -O3
LDLAGS = -O3
OBJS = main_file.o file1.o file2.o

all: hellomake

hellomake: $(OBJS)
    $(CXX) $(OBJS) -o hellomake $(LDLAGS)

main_file.o: main_file.cpp file1.h file2.h
    $(CXX) -c main_file.cpp $(CXXFLAGS)
.........................
```

Припустимими є також порожні змінні, наприклад:

```
CFLAGS =
```

Змінні, визначені автоматично

Ряд змінних присутні завжди:

- $@ — ім'я файлу цілі.
- $* — ім'я файлу цілі без розширення.
- $< — перша з залежностей.
- $^ — всі залежності, розділені пробілами, без повторів.
- $+ — те ж, з повторами.
- $? — список всіх залежностей, що новіші за ціль.

З їх використанням можемо переписати файл так:

```
CXX = g++
CXXFLAGS = -O3
LDLAGS = -O3
OBJS = main_file.o file1.o file2.o
PRG_NAME =  hellomake

all: $(PRG_NAME)

$(PRG_NAME): $(OBJS)
    $(CXX) $^ -o $@

main_file.o: main_file.cpp file1.h file2.h
    $(CXX)  -c $(CXXFLAGS) $<
file1.o: file1.cpp  file1.h
    $(CXX)  -c $(CXXFLAGS) $<
file2.o: file2.cpp  file2.h
    $(CXX)  -c $(CXXFLAGS) $<

.PHONY : clean
clean:
    rm -rf *.o $(PRG_NAME)
```

Зауважте, у попередньому прикладі всі команди для файлів *.cpp однакові. Можна позбутися такого дублювання, правда, ціною визнання, що всі об'єктні файли залежать від всіх заголовків (*.h). Зробити це можна за допомогою шаблонних правил:

```
CXX = g++
CXXFLAGS = -O3
LDLAGS = -O3
OBJS = main_file.o file1.o file2.o
PRG_NAME =  hellomake
HEADERS = file1.h file2.h              # <============

all: $(PRG_NAME)

$(PRG_NAME): $(OBJS)
    $(CXX) $^ -o $@

%.o: %.cpp $(HEADERS)                    # <============
    $(CXX) -c $(CXXFLAGS) $<           # <============

.PHONY : clean
clean:
    rm -rf *.o $(PRG_NAME)
```

Virtual Path — VPATH

- VPATH — змінна, яка служить для вказання шляхів, де шукати залежності.

```
VPATH = src include
```

- vpath використовуються, щоб конкретизувати, де шукати залежності різних типів:

```
vpath %.cpp src
vpath %.h include
```

Підстановки

Детально не розглядатимемо поки, лише як реклама. Якщо потрібно просто скомпілювати всі *.cpp файли в поточній директорії, можна зробити щось таке:

```
SOURCES = $(wildcard *.cpp)
# чи
SOURCES = $(wildcard src/*.cpp)
# І тоді:
OBJS = $(subst .cpp,.o,$(SOURCES))
# або
OBJS = $(SOURCES:.cpp=.o)
```

Якщо ж бажано контроль зберегти, то можна так:

```
SOURCES=main_file.cpp file1.cpp file2.cpp
OBJS=$(SOURCES:.cpp=.o)
```

Див. також `https://www.gnu.org/software/make/manual/html_node/Text-Functions.html`, зокрема – filter, filter-out, sort.

Підстановки шляхів

Забрати символ / в кінці шляху (наприклад, з /home/path/prj/) можна якось так:

```
$(patsubst %/,%,$(SOMEDIR))
```

Фокус базується на тому, що символ % означає "будь-що". Звичайно, можна використовувати і для цікавіших підстановок:

```
OBJS = $(patsubst src/%.c,obj/%.o,$(SRC))
чи й:
OBJS = $(SOURCES: src/%.c=obj/%.o)
```

GNU Make: "Ліниве" та негайне присвоєння

Змінна створена так:

```
CXX = g++
```

виконувати те, що праворуч від "=" за кожного використання. Часто це не те, що потрібно. Наприклад, такий код (див. далі):

```
SOURCES = $(wildcard *.cpp)
```

може бути занадто повільним або мати невірну логіку за постійного оновлення вмісту SOURCES. Тоді **GNU Make** дозволяє скористатися оператором ":=", який виконує негайне присвоєння:

```
CFLAGS := -O3
```

Інші цікаві розширення, це: присвоїти змінну, якщо її ще не існує:

```
CFLAGS ?= -O3
```

та додати до вмісту:

```
CFLAGS += -O3
```

Див., скажімо, `https://stackoverflow.com/questions/448910/what-is-the-difference-between-the-gnu-makefile-variable-assignments`

GNU Make: умовні конструкції

- Приклад умовної конструкції із офіційної документації:

```
libs_for_gcc = -lgnu
normal_libs =

foo: $(objects)
ifeq ($(CC),gcc)
        $(CC) -o foo $(objects) $(libs_for_gcc)
else
        $(CC) -o foo $(objects) $(normal_libs)
endif
```

- Детальніше див. https://www.gnu.org/software/make/manual/html_node/Conditionals.html.

GNU Make: shell substitution

- GNU Make вміє здійснювати підстановку виводу команди, виконаної оболонкою (shell):

```
contents := $(shell cat foo)
files := $(shell echo *.c)
SOURCES := $(shell find src -name "*.cpp")
# != -- альтернативний синтаксис:
SOURCES != find src -name "*.cpp"
```

- Не зловживайте цим – запуск оболонки достатньо повільний.
- Детальніше див. `https://www.gnu.org/software/make/manual/html_node/Shell-Function.html`.

Неявні правила

Make знає, як будувати цілий ряд файлів. Тому, якщо написати

```
main_file.o: main_file.cpp
file1.o: file1.cpp
file2.o: file2.cpp
```

Він сам викличе щось типу:

```
 $(CXX) -c $(CXXFLAGS) $<
```

Тобто, врахує вміст CXX, CXXFLAGS (CC і CFLAGS для C).

Подивитися поточний список неявних правил можна так:
`--print-data-base`.

GNU Make: Автоматичні залежності

Якщо користуватися парою GNU make та GCC, є можливість автоматично враховувати неявні залежності між файлами проекту, такі як включення заголовків.

```
<.... Те, що і раніше ....>
all: $(PRG_NAME)

# Включити всі наявні залежності
-include $(OBJS:.o=.d)

$(PRG_NAME): $(OBJS)
    $(CXX)  $(LDLAGS) $^ -o $@

%.o: %.cpp
    $(CXX) -c $(CXXFLAGS) $<
    $(CXX) -MM -MP -MT $@  -MF $*.d $(CXXFLAGS) $<
# Рядок вище генерує інформацію про залежності

.PHONY : clean
clean:
    rm -rf *.o *.d $(PRG_NAME)
```

Фінальний штрих — наводимо порядок

Зараз в директорії проекту купа робочих файлів. Можна навести з ними порядок. (Директорії bin і obj мають вже існувати на момент запуску компіляції!):

```
CXX = g++
CXXFLAGS = -O3
LDLAGS = -O3
PRG_NAME =  hellomake
BIN_DIR = bin
OBJ_DIR = obj
SOURCES = main_file.cpp file1.cpp file2.cpp
OBJS = $(patsubst %.cpp,$(OBJ_DIR)/%.o,$(SOURCES))

all: $(BIN_DIR)/$(PRG_NAME)
```

Продовження на наступному слайді.

```
# Включити всі наявні залежності
-include $(OBJS:.o=.d)

$(BIN_DIR)/$(PRG_NAME): $(OBJS)
    $(CXX)  $(LDLAGS) $^ -o $@

$(OBJ_DIR)/%.o: %.cpp
    $(CXX) -c $(CXXFLAGS) $<  -o $@
    $(CXX) -MM -MP   -MT $@  -MF $(OBJ_DIR)/$*.d $(CXXFLAGS) $<
# Рядок вище генерує інформацію про залежності

.PHONY : clean
clean:
    rm -rf $(OBJ_DIR)/*.o $(OBJ_DIR)/*.d $(BIN_DIR)/$(PRG_NAME)
```

Debug makefile-скриптів (DRAFT!)

- https://habr.com/ru/post/534128/ та https://habr.com/ru/post/534304/.

Детальніше див., наприклад:

- `http://scottmcpeak.com/autodepend/autodepend.html`
- `http://mad-scientist.net/make/autodep.html`
- Офіційна документацію GNU make: `https://www.gnu.org/software/make/manual/html_node/Automatic-Prerequisites.html`
- GCC: `http://gcc.gnu.org/onlinedocs/gcc-4.8.2/gcc/Preprocessor-Options.htm` — значення "-MM", "-MP", "-MT", "-MF". `http://stackoverflow.com/questions/97338/gcc-dependency-generation-for-a-different-output-directory`
- `http://mad-scientist.net/make/multi-arch.html`
- `http://stackoverflow.com/questions/5178125/gnu-make-how-to-get-object-files-in-separate-subdirectory`
- `https://devhints.io/makefile` – коротка шпаргалка.
- `http://make.mad-scientist.net/papers/multi-architecture-builds/` – деякі цікаві трюки є і тут.
- "Self-documenting Makefiles" – цікавий есей про "безпечне" документування makefile.